Validation Protocol — how we prove the code does what we say

Version 1.0 · 2026-09-18 · DRAFT for Reza's review. Part of the Research Integrity Charter (rules R5, R6).

Principle: a result may be reported only at the validation tier its claim requires. "The code ran without errors" is not validation. Each check compares our output with something we did not produce ourselves: an exact mathematical answer, code written by other people, or a control with a known outcome.

The four tiers

Tier 1 — Is the math computed correctly?

Check	What it does	Typical pass criterion
Analytic controls	Run the code on shapes whose answer is known exactly: the unit sphere's spectrum is l(l+1), multiplicity 2l+1	< 1% error on low modes
Invariance tests	The same shape scaled, rotated, re-triangulated, or bent without stretching must give the same descriptor	Differences at the level of discretization noise
Independent reference implementations	Compare with code written by *other people*: LaPy (Reuter group, the originators of shapeDNA), gptoolbox (MATLAB), robust-laplacian, etc.	Pre-registered tolerance
Reproduce published results	Compute a descriptor on a public benchmark and match numbers or rankings a paper reports	Match within the reported precision

Tier 2 — Does the data survive the pipeline intact?

Check	What it does
Provenance trace	Each object traced from source file to final result, with how much geometry was kept and what was changed (logged, with counts)
Regression tests	Saved reference outputs. Any change after a code edit gets flagged, which catches silent errors like the 381-object keep-largest bug
Visual spot-checks	Before/after renders of a random sample (not a hand-picked one)

Tier 3 — Do the maps and statistics mean anything?

Check	What it does	Guards against
Positive controls	Synthetic shapes with *known* differences (ShapeForge genes). The analysis must recover the known structure.	A method blind to real structure
Negative controls	Shuffle the descriptor values, feed in random noise, or permute labels. Any "structure" must disappear.	Seeing patterns in noise
Permutation tests	Is observed cluster crispness (silhouette) beyond what, say, 1000 shuffles produce?	Chance clusters
Stability	Re-run with different random seeds and on 80% subsamples. Conclusions must hold.	Fragile results that flip between runs
Held-out evaluation	Any parameter chosen by searching is scored on data *not* used in the search	Cherry-picking; the multiple-comparisons problem

Tier 4 — Can a human defend it?

Check	What it does
Plain-language methods	Every method explained without jargon, with a MATLAB equivalent where one is feasible
Parameter registry	Every setting with its value, provenance, source status and sensitivity (template)
Independent human review	A qualified expert reviews the methods and code before publication

Which tier a claim needs

Claim type	Minimum tiers
Internal exploration or a pretty picture	1 (core operator) + labelled "exploratory"
"Descriptor X separates category Y" (in a talk or poster)	1, 2, 3 (negative control + stability)
Anything in a paper, thesis or grant as a *result*	1–4, all of them

How to run a check (procedure)

1. **Pre-register.** Before running, write down the question, comparisons, test cases (chosen by a fixed rule, not by hand) and pass criteria in `PREREGISTRATION_<name>.md`. **Commit it to git** so the timestamp proves the criteria came first.
2. **Run** the scripts. Apply the criteria **mechanically**, in code.
3. **Report** in `REPORT_<name>.md` + `.html`: the overall verdict, a table per criterion, a figure, and a section on "what this does and does not establish".
4. **Post-hoc observations** go in a separate, clearly labelled section. They never change the verdict.
5. **Failures are reported as failures.** Criteria are never relaxed after the fact. A failed check creates a task: investigate, fix, then re-run with a *new* pre-registration.

6. **Commit** the results, and record the commit hash anywhere the result is cited.

Status board — Lab_3d (as of 2026-09-18)

Tier / check	Status	Evidence
T1 analytic: LB sphere, scale invariance, zero mode, anisotropic reduces to cotangent, Hodge sphere, mass-orthonormality, HKS sanity, Weyl's law	✅ done (9/9)	`gso_shape_explorer/VALIDATION.html` (not pre-registered)
T1 reference: shapeDNA vs LaPy + gptoolbox (k ≤ 99)	✅ **done, pre-registered, 5/5 pass**	`Lab_3d/validation/REPORT_reference_check.html` ; git `39ae3d4` → `ccfb5f5`
T1 reference: k up to 1000 (the full shapeDNA used)	⬜ to do	Motivated by a post-hoc observation (gap grows with index)
T1 reference: derived descriptors (HKS, WKS, GPS, biharmonic, anisotropic) against papers' conventions or authors' code	⬜ to do	The parameter conventions are still UNVERIFIED
T1 invariance (re-triangulation, isometric bending)	⬜ to do	
T2 provenance trace: GSO repair (weld, keep-largest, watertight)	🟡 partial	`gso_repair_triage.csv` logs geometry kept per object; not yet a formal report
T2 regression tests	⬜ to do	
T3 positive control: ShapeForge genes vs descriptors	⬜ to do	ShapeForge audit (9/9) validates the *generator*, not the descriptors
T3 negative controls + permutation test on clusters	⬜ to do	**High priority:** the "best normalization" was chosen by maximizing cluster crispness on the same data
T3 stability (seeds, subsamples)	⬜ to do	
T3 held-out re-evaluation of the normalization search and the THINGS constraint search	⬜ to do	Both are TUNED choices, currently at risk of cherry-picking
T4 parameter registry	🟡 pre-filled	`PARAMETER_REGISTRY.xlsx` ; Reza to review
T4 independent human review	⬜ to do	Reviewer to be named

Bottom line today: the Laplace–Beltrami foundation is independently verified. Everything built on top of it — derived descriptors, normalization choice, clusters, maps — remains **exploratory** until Tiers 1 (derived), 3 and 4 are done.